

Deep Learning-Based Autonomous Road-Following for JetBot

Robotics II — Project Report

Team Report - Piotr Franc, Jakub Adamski, Piotr Foltyniewicz, Wiktor Talarek, Daniel Skalski

June 11, 2026

1 Introduction

The report describes the process of creating software for autonomous driving for a jetbot that was a task for robotics class. We describe our methodology and issues that we encountered.

2 System Pipeline

2.1 Dataset Acquisition

The training dataset was sourced from a repository provided by PUT, consisting of video frames recorded by a JetBot navigating the track. Multiple sessions were collected with different tracks and lighting, and during the training process we used data augmentation to make the model more robust.

2.2 Dataset Annotation

Although the dataset included pre-recorded coordinates, we chose to re-annotate the images manually using a custom Python annotation tool (`annotation_script.py`). The tool displayed each frame interactively and allowed the annotator to click directly on the image to place a target point representing where the robot should steer towards. In total we annotated 1200 images across different datasets to test different behaviours of the robot.

2.3 Model Training

Training was performed using PyTorch with a pretrained ResNet-18 backbone. We also tried MobileNet when we wanted to speed up inference but in the end we decided to stick with resnet as the inference and error were more or less comparable.

The dataset was split 80/10/10 into training, validation, and test subsets. The model was optimised using Adam over 70 epochs with a batch size of 8, minimising mean squared error (MSE) between predicted and true coordinates. After training, the model's state dictionary was exported in PyTorch's legacy serialisation format using `_use_new_zipfile_serialization=False` to ensure compatibility with older JetBot software stacks.

2.4 On-Robot Execution

The trained model was uploaded to the JetBot and loaded in the live demo notebook. At runtime, each frame captured by the JetBot camera was preprocessed and passed through the model on the GPU in half-precision (`float16`) via `torch.jit.trace` for reduced latency. The predicted (x, y) coordinate was converted to a steering angle using:

$$\theta = \arctan 2(x - 0.5, 1 - y)$$

We tested different functions for transforming the goal point to angle but ended up using the simple `arctan`.

The key tunable parameters exposed as sliders were `SPEED_GAIN`, `STEERING_GAIN`, `STEERING_DGAIN`, and `STEERING_BIAS`. We've set speed gain to 0.28 and steering gain to around 0.09 (I don't remember the exact numbers, and probably they'd vary anyway from jetbot to jetbot).

3 Challenges Encountered

The project involved several significant practical obstacles, which are described below.

Hardware variance. The JetBots available in the lab were not uniform. Some units lacked a functioning CUDA device, making GPU-accelerated inference impossible on those robots. This made the robot's reaction time too slow for it to complete the track in reasonable time. Additionally, one JetBot had a mechanically defective wheel, which meant we had to reset all the progress we had on that particular machine.

Library version incompatibility. Several robots ran older versions of PyTorch and JetPack that could not load model files saved in PyTorch's default (new) zip-based format. This required re-exporting the trained state dictionary using the legacy serialisation flag

```
_use_new_zipfile_serialization=False
```

before transferring it to the robot.

Camera lifecycle issues. The original live demo design relied on JetBot's `camera.observe()` callback and `camera.stop()` to drive the inference loop. In practice, if the Jupyter notebook was shut down incorrectly (e.g. by closing the browser tab rather than executing the stop cell), the camera process would be left running in a zombie state that persisted across reconnections. The only recovery was a full JetBot reboot. The observe loop was also slow and buggy and we immediately felt the difference when we swapped to a simple while loop. This gave us full control and fixed most of our problems.

Annotation quality and repeated labelling. The robot's behaviour proved highly sensitive to annotation quality. After deploying an initial model, we observed that the JetBot would tilt excessively and frequently leave the track on sections with dense turns. We fixed that by making the robot 'skip' this part by annotating such places as a straight road, which improved the robots speed by a big margin.